



WIPRO CAPSTONE PROJECT

AppVerse AI

Smart App Marketplace & Review Intelligence Platform



Complete Technical Documentation

Full-Stack Capstone Project — Spring Boot & React

Author: Gopidi Anugna Reddy | Version 1.0.0

June 2026



Table of Contents

1. Introduction & Project Overview
2. System Architecture
3. Database Design & ER Diagram
4. UML Class Diagram
5. Use Case Diagram
6. Data Flow Diagram (DFD)
7. Frontend Technical Documentation
8. Backend Technical Documentation
9. Key Workflow Sequences
10. Module Descriptions
11. Project Screenshots
12. Deployment & Submission Guidelines
13. Conclusion & Future Enhancements
14. Submitted By

1. Introduction & Project Overview

AppVerse AI is a full-stack web application that recreates the core experience of an app marketplace such as the Google Play Store, extended with AI-driven intelligence. The platform allows everyday users to discover, download, and review mobile applications; lets registered developers publish and manage their own apps; and gives administrators full control over what is allowed to go live on the platform. What sets AppVerse AI apart from a conventional CRUD marketplace is that several of its core features are powered by the Google Gemini API: every review submitted to the platform is automatically analyzed for sentiment and checked for signs of being fake, users can ask a conversational AI assistant (AppBot) to recommend apps in natural language, and the recommendation engine adapts its strategy based on a user's download history.

The project was built as a single Spring Boot backend application secured end-to-end with JWT authentication, paired with a React frontend that renders three distinct experiences depending on the signed-in user's role. The system follows a clean layered architecture (Controller → Service → Repository → Entity) and applies recognized object-oriented design patterns — Strategy, Factory, Observer, and Facade — to keep the recommendation logic, app-creation logic, and review-moderation logic loosely coupled and easy to extend.

1.1 Objectives

The project was designed around the following goals:

- Build a secure, role-based marketplace where users, developers, and admins each get a purpose-built interface.
- Implement JWT-based authentication strengthened with Gmail OTP verification at registration, so that every account is tied to a verified email address.
- Integrate a real generative-AI service (Google Gemini) for three distinct features: review sentiment analysis, a conversational marketplace assistant, and natural-language search support.
- Apply a moderation workflow so that no app or review reaches end users without passing through an approval or visibility check.
- Demonstrate sound software design by applying multiple Gang-of-Four design patterns within a real, working codebase rather than as isolated examples.
- Provide developers and administrators with usable analytics — download trends, revenue, engagement, and category insights — derived from the same underlying data the marketplace runs on.

1.2 Platform Roles & Capabilities

Table 1: Platform Roles and Capabilities

Role	Capabilities
USER	Register with Gmail OTP verification, browse and search the marketplace, download apps, write star ratings and reviews, chat with the AppBot AI assistant, and receive AI-personalized app recommendations.
DEVELOPER	Create and submit new apps for review, publish version updates with release notes, track per-app and platform-wide download analytics, and view AI sentiment trends across their own apps' reviews.
ADMIN	Approve or reject pending app submissions, moderate reviews flagged as fake or low-quality, and monitor platform-wide statistics from a dedicated admin dashboard.

1.3 Key Features

- **AI-Powered Review Intelligence:** every submitted review is passed through a sentiment classifier (POSITIVE / NEUTRAL / NEGATIVE), given a predicted star rating derived purely from its text, and screened by a fake-review heuristic before it is shown publicly.
- **AppBot Conversational Assistant:** a floating chat widget, visible across the marketplace, lets users ask natural-language questions (“suggest a few apps for learning”) and receive Gemini-generated, context-aware app suggestions with direct links.
- **Strategy-Based Recommendations:** the recommendation engine swaps between a Trending strategy and a Top-Rated strategy depending on whether the requesting user has any download history, avoiding a cold-start problem for new users.
- **JWT Authentication with Gmail OTP:** registration requires verifying a one-time passcode sent to the user's email before an account becomes active, adding a real identity check on top of stateless token authentication.
- **Full Moderation Lifecycle:** apps move through PENDING → APPROVED / REJECTED states, and reviews carry their own independent moderation status, so admins can manage app quality and review quality separately.
- **Role-Based Access Control:** three roles (USER, DEVELOPER, ADMIN) each see a different navigation structure and are restricted at both the frontend route level and backend endpoint level.
- **Developer & Admin Analytics:** download trends over time, revenue by app, free-vs-paid distribution, and per-app engagement scores are all computed from live data and rendered as charts in the console.

1.4 Document Scope

This document provides complete technical documentation for the AppVerse AI platform, covering system architecture, database design, UML and data-flow modelling, frontend and backend implementation detail, the key end-to-end workflows, and screenshots of the working application. It is intended for academic evaluators, mentors, and any developer who needs to understand how the system is designed and why specific implementation choices were made. The submission package accompanying this document

additionally includes the full source code, database scripts, a Postman collection for API validation, and supporting screenshots, as detailed in Section 12.

2. System Architecture

AppVerse AI is built as a single, independently deployable Spring Boot application rather than a microservices cluster. This keeps the system simple to run, test, and grade, while the internal code is still organized into clean, independent layers so the same separation of concerns a microservices system would have at the network level exists here at the package level.

2.1 Layered Architecture

Every request passes through the same fixed pipeline: the Controller layer exposes REST endpoints and performs request validation; a JWT filter sits ahead of the controllers and authenticates every incoming request; the Service layer holds all business logic, including sentiment analysis, rating prediction, fake-review detection, and recommendation strategy selection; the Repository layer (Spring Data JPA) abstracts all database access; and the Entity layer maps directly onto the MySQL schema. The only external dependency outside this pipeline is the Google Gemini API, called from the service layer whenever an AI-driven feature is invoked.

AppVerse AI — System Architecture

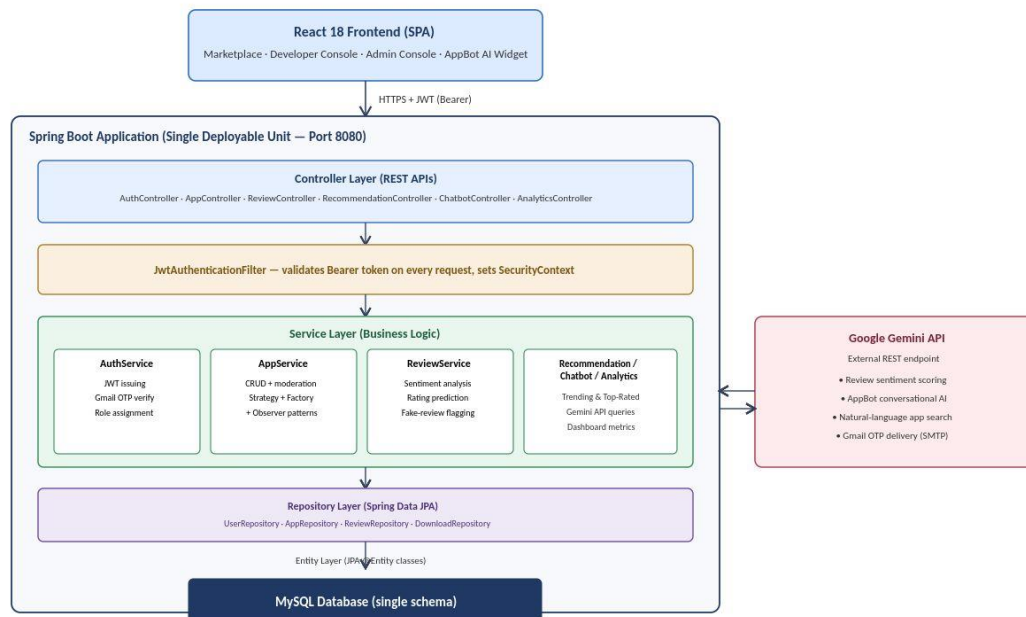


Fig. 5.2: Layered architecture of the AppVerse AI Spring Boot application. A single deployable backend serves three role-based React experiences and calls the Gemini API for AI-driven features.

Fig. 2.1: Layered architecture of the AppVerse AI Spring Boot application.

2.2 Technology Stack

Table 2: Technology Stack

Layer	Technology	Purpose
Frontend	React 18	Component-based UI framework for all three role-based experiences
Frontend	Fetch / Axios-style API calls	Communication with the Spring Boot REST API
Frontend	React Context / State Hooks	Client-side session and role-based UI state
Backend	Spring Boot 3.x	Core application framework
Backend	Spring Security + JWT	Stateless authentication and route protection
Backend	Spring Data JPA / Hibernate	ORM and repository-based database access
Backend	Lombok	Boilerplate-free entities via @Getter/@Setter/@Builder
Database	MySQL	Single relational schema for all entities
AI	Google Gemini API	Sentiment analysis, AppBot chatbot, recommendation support
Email	Gmail SMTP (JavaMail)	OTP delivery during registration
Build Tool	Maven	Dependency management and build lifecycle
IDE / Tooling	Spring Tool Suite 4	Backend development environment

2.3 Design Patterns Applied

Rather than treating design patterns as an academic checkbox, AppVerse AI applies four patterns to genuinely simplify extension points in the codebase:

- **Strategy Pattern** — RecommendationContext switches between TrendingRecommendationStrategy and TopRatedRecommendationStrategy depending on whether the requesting user has prior download history, without the controller or calling service needing to know which concrete strategy is active.
- **Factory Pattern** — used in app creation to construct the correct initial entity state (default status, default visibility flags, timestamps) consistently across every code path that creates a new App.
- **Observer Pattern** — used to decouple side effects (such as updating an app's average rating and review count) from the act of submitting a review, so ReviewService does not need to know about every consumer of a new review event.
- **Facade Pattern** — the AnalyticsService and RecommendationServiceImpl expose simplified, single-method entry points (getCategoryInsights(), getPersonalizedRecommendations()) that internally coordinate multiple repositories and calculations, hiding that complexity from the controllers.

3. Database Design & ER Diagram

AppVerse AI uses a single MySQL schema shared by every entity in the application. Five core tables capture the entire domain: users, apps, app_versions, reviews, and downloads, supported by enumerated reference types for role, category, app status, sentiment, and moderation status.

3.1 Core Entities

Table 3: Core Database Tables

Table	Key Fields	Purpose
users	user_id (PK), name, email, password, role, otp_verified	Stores account credentials, role assignment, and OTP verification state
apps	app_id (PK), name, category, price, average_rating, status, developer_id (FK)	Stores marketplace listings created by developers
app_versions	version_id (PK), version_number, release_notes, app_id (FK)	Tracks the version history of each published app
reviews	review_id (PK), rating, comment, sentiment, predicted_rating, is_flagged, moderation_status, user_id (FK), app_id (FK)	Stores user reviews along with AI-derived sentiment and moderation metadata
downloads	download_id (PK), downloaded_at, user_id (FK), app_id (FK)	Tracks unique user-app download events used for analytics and recommendations

3.2 Entity-Relationship Diagram

AppVerse AI — Entity-Relationship Diagram

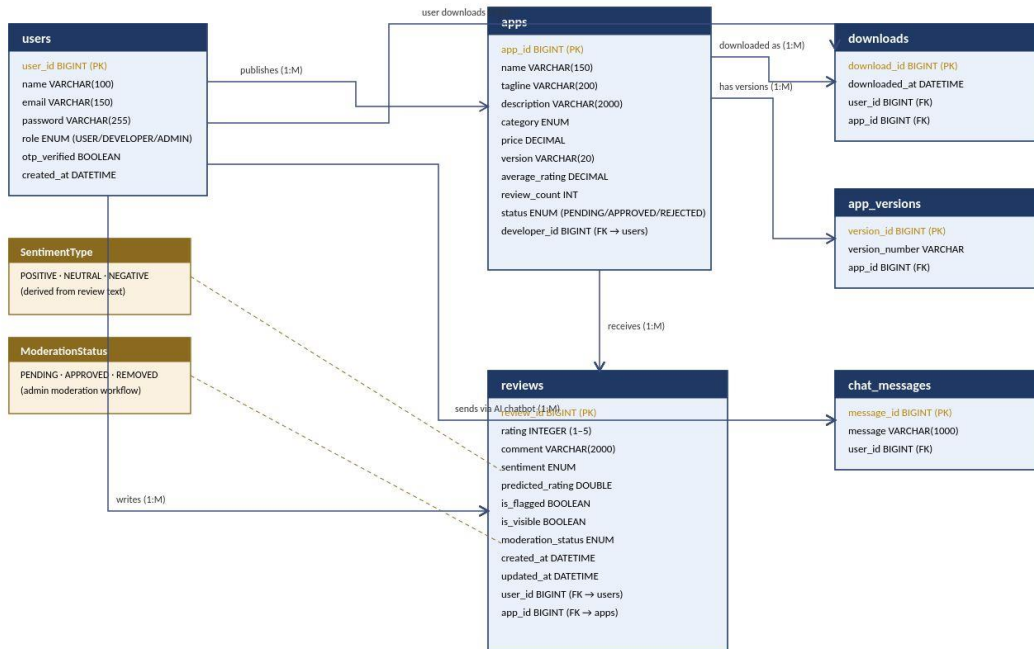


Fig. 5.1: Core relational schema (single MySQL database). Solid arrows denote foreign-key relationships; dashed lines denote enumerated-type references.

Fig. 3.1: Entity-Relationship Diagram for the AppVerse AI MySQL schema.

3.3 Key Relationships

- **User → App (1:M)** — a developer (a User with role DEVELOPER) can publish many apps; each app has exactly one developer.
- **App → AppVersion (1:M)** — each app maintains a full version history, with release notes captured per version.
- **User → Download (1:M)** and **App → Download (1:M)** — downloads form a many-to-many bridge between users and apps, used both for the download counter and as the input signal for the recommendation engine.
- **User → Review (1:M)** and **App → Review (1:M)** — every review belongs to exactly one user and one app; an app's `average_rating` and `review_count` are recalculated from its visible reviews whenever a review is added, edited, or moderated.
- **User → ChatMessage (1:M)** — messages exchanged with the AppBot AI assistant are associated with the user who sent them.

4. UML Class Diagram

The class diagram below shows the core domain classes exactly as implemented in the Spring Boot entity layer, along with the enumerations they depend on and the Strategy pattern interface used by the recommendation engine.

AppVerse AI — UML Class Diagram

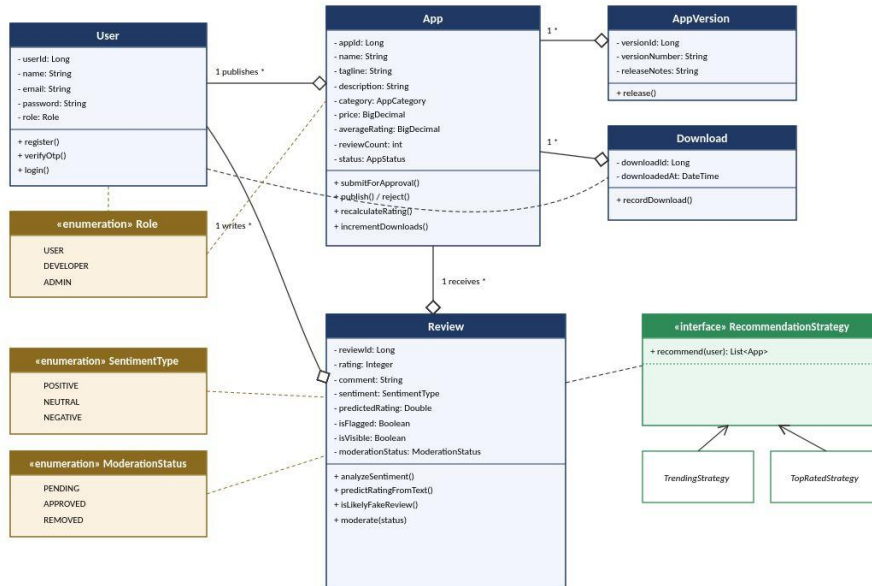


Fig. 6.1: UML class diagram showing core domain classes, enumerations, and the Strategy pattern used for app recommendations.
Legend: ◆ = composition / ownership —··· (dashed, hollow arrow) = interface realization — (dashed line) = dependency / enum reference

Fig. 4.1: UML class diagram showing core domain classes, enumerations, and the Strategy pattern.

4.1 Notes on the Design

- **User**, **App**, and **Review** are the three central entities; all other classes either support them (**AppVersion**, **Download**) or categorize them (the enumerations).
- **SentimentType** and **ModerationStatus** are populated by the backend automatically — never set directly by the client — reinforcing that AI-derived and moderation-derived fields are system-owned data.
- **RecommendationStrategy** is implemented by **TrendingStrategy** and **TopRatedStrategy**, illustrating the Strategy pattern described in Section 2.3.

5. Use Case Diagram

The use case diagram identifies the three human actors in the system (User, Developer, Admin) and the one external system actor (the Google Gemini API), and maps each actor to the functionality they can trigger.

AppVerse AI — Use Case Diagram

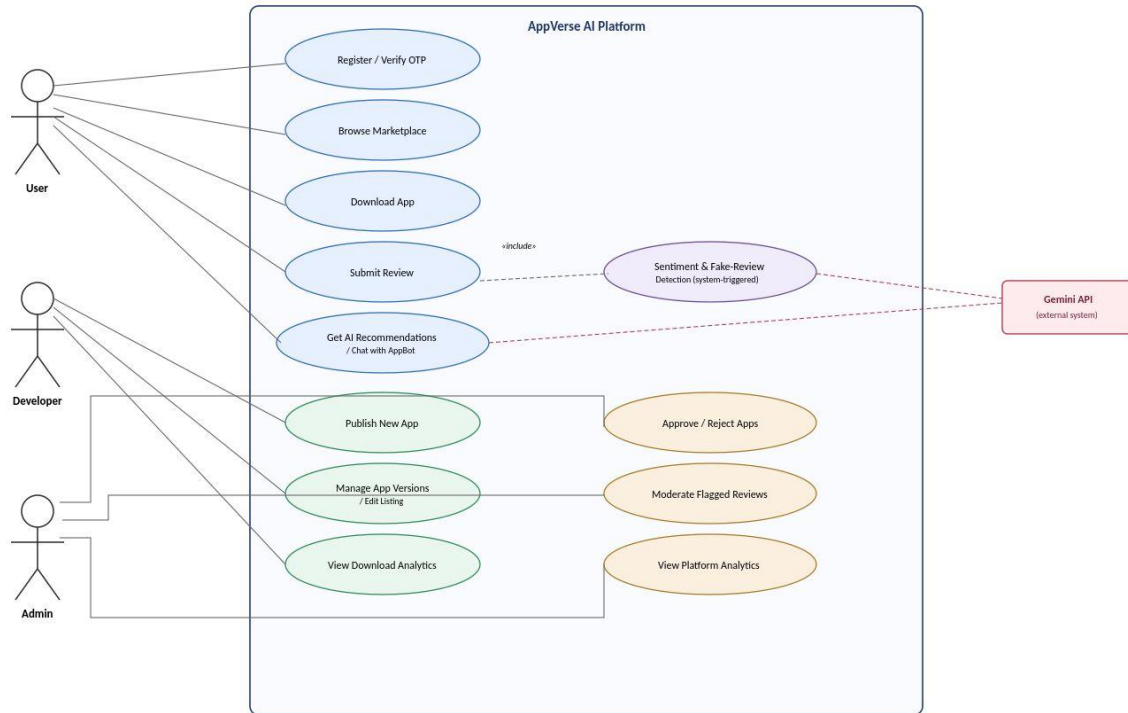


Fig. 6.2: Use case diagram showing the three platform actors (User, Developer, Admin) and the external Gemini AI system that supports sentiment analysis, recommendations, and the AppBot chatbot.

Fig. 5.1: Use case diagram showing platform actors and their associated use cases.

5.1 Notes on the Design

- Submit Review includes Sentiment & Fake-Review Detection as a mandatory sub-flow — a review can never be saved without first being scored, modelled here using the UML «include» relationship.
- Get AI Recommendations / Chat with AppBot and the sentiment detection use case both depend on the external Gemini API actor, shown with dashed association lines since this is a system-to-system call rather than a user-triggered action.
- Developer and Admin use cases are kept deliberately separate even though both relate to apps: a Developer can manage their own apps' versions and analytics, but only an Admin can approve, reject, or moderate content platform-wide.

6. Data Flow Diagram (DFD)

The Level-1 Data Flow Diagram below traces how data moves between the platform's external entities (User / Developer / Admin and the Gemini API), its six core processes, and the single shared MySQL data store.

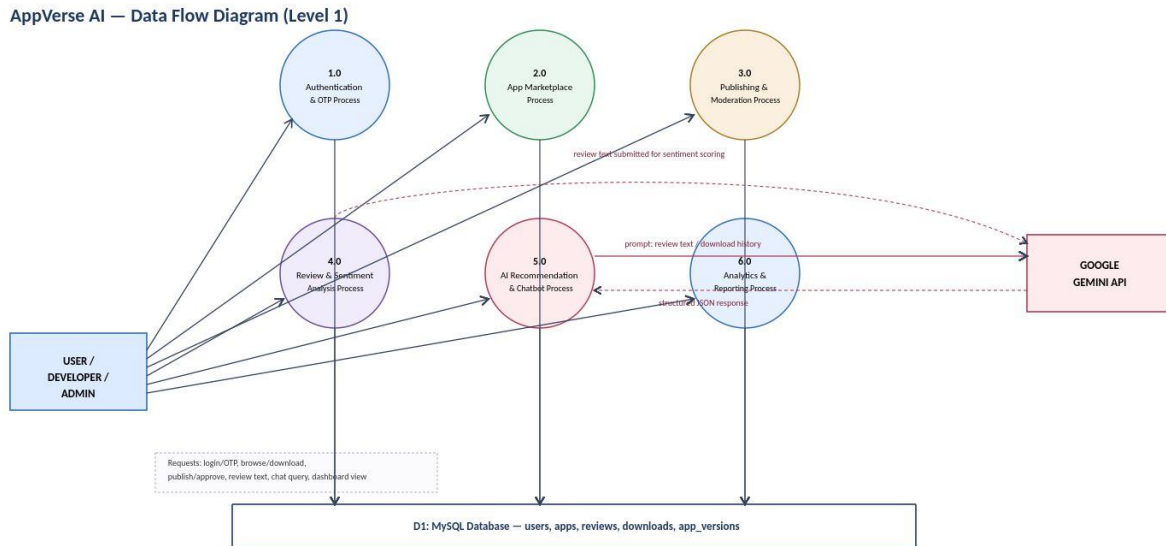


Fig. 6.3: Level-1 Data Flow Diagram showing the six core processes, the single shared database, and the external Gemini AI dependency.

Fig. 6.1: Level-1 Data Flow Diagram for AppVerse AI.

6.1 Process Summary

Table 4: DFD Process Descriptions

Process	Description
1.0 Authentication & OTP	Validates credentials, issues JWTs, and verifies Gmail OTP codes at registration
2.0 App Marketplace	Handles browsing, search, filtering, and download requests
3.0 Publishing & Moderation	Handles developer app submission and admin approve/reject decisions
4.0 Review & Sentiment Analysis	Accepts review submissions, calls Gemini for sentiment scoring, and applies fake-review heuristics
5.0 AI Recommendation & Chatbot	Selects a recommendation strategy and serves AppBot conversational queries via Gemini

Process	Description
6.0 Analytics & Reporting	Aggregates download, revenue, and engagement data for developer and admin dashboards

7. Frontend Technical Documentation

7.1 Project Structure

The React frontend is organized by role, mirroring the backend's access-control boundaries. Each role has its own set of pages, while shared concerns — authentication state, API calls, and the AppBot widget — live in common modules used across all three experiences.

- Authentication pages: Login, Register (with OTP verification step)
- User pages: Marketplace (search, filters, AI recommendations), App Details (reviews, ratings, similar apps)
- Developer pages: Developer Console (My Apps, Analytics tabs), Publish/Edit App form
- Admin pages: Admin Dashboard (Pending Apps, Flagged Reviews tabs)
- Shared: AppBot AI chat widget (floating, available on every authenticated page)

7.2 Authentication Flow

On login, the frontend stores the JWT returned by the backend and attaches it as a Bearer token to every subsequent API request. The signed-in user's role is decoded from the session state and used to decide which navigation links and pages are rendered — a USER never sees the Console or Admin tabs, and a DEVELOPER never sees the Admin tab, even though the underlying API additionally re-checks role on every protected endpoint.

7.3 Key Frontend Behaviors

- AppBot Widget: a persistent chat interface (“Powered by Gemini · Always online”) that accepts free-text questions and returns recommended apps as clickable cards with a direct “View” link into the marketplace listing.
- AI Sentiment Badges: each review displayed on an app's detail page shows a colored sentiment badge (POSITIVE / NEUTRAL / NEGATIVE) returned directly from the backend, with no client-side recomputation.
- Similar Apps Panel: app detail pages show an AI-curated “Similar Apps” list in the page sidebar, separate from the main recommendation engine used on the marketplace home.
- Developer Console Tabs: My Apps (CRUD + status) and Analytics (downloads trend, revenue by app, free vs. paid split, engagement table) are rendered as separate tabs within the same console, both reading from the same developer-scoped API.
- Admin Approval Workflow: the Admin Dashboard surfaces a single pending-apps queue with Preview, Approve, and Reject actions; once approved, an app becomes visible in the public marketplace within the same request cycle.

8. Backend Technical Documentation

8.1 Service Architecture

All business logic lives in the Service layer, with one service per major domain area. Controllers remain thin — they validate input shape and delegate immediately to a service method.

Table 5: Core Backend Services

Service	Responsibility
AuthService	Registration, password hashing, JWT issuance, Gmail OTP generation and verification
AppService	App CRUD, submission/approval/rejection workflow, average rating recalculation
ReviewService	Review CRUD, rule-based sentiment analysis, rating prediction from text, fake-review flagging, moderation
RecommendationService	Strategy selection (Trending vs. Top-Rated), personalized recommendations, trending and category insight analytics
ChatbotService	Receives AppBot queries, builds Gemini prompts, parses structured JSON responses into app suggestions
AnalyticsService	Aggregates download counts, revenue, free-vs-paid splits, and engagement scoring for dashboards

8.2 Security & JWT Flow

AppVerse AI uses stateless JWT authentication. On successful login, the backend issues a signed token containing the user's ID and role. A `JwtAuthenticationFilter` intercepts every incoming request, validates the token's signature and expiry, and populates the Spring Security context with the authenticated user before the request reaches any controller. Endpoints that require a specific role (for example, the admin approval endpoints) are additionally annotated to reject any token that does not carry the required authority.

- Passwords are never stored in plain text; they are hashed before persistence.
- Registration requires a Gmail OTP step: a one-time code is emailed to the address provided, and the account is not marked `otp_verified` until the correct code is submitted back.
- Each request thread maintains its own isolated authentication context, so concurrent requests from different users cannot interfere with one another.

8.3 AI Integration: Sentiment Analysis & Fake-Review Detection

`ReviewService.analyzeSentiment()` inspects the review's rating and free-text comment and classifies it as POSITIVE, NEUTRAL, or NEGATIVE. `predictRatingFromText()` independently estimates what star rating the

text itself implies, which is then compared against the rating the user actually selected.

`isLikelyFakeReview()` flags a review as suspicious when the comment is unusually short, the predicted rating diverges sharply from the submitted rating, or the comment contains repetitive, low-effort text patterns. Reviews flagged this way are not deleted; they are routed into the Admin Dashboard's Flagged Reviews queue for a human moderation decision, with `isVisible` set to false until an admin acts.

8.4 AI Integration: Recommendations & AppBot

`getPersonalizedRecommendations()` first checks whether the requesting user has any prior downloads. If they do not (a cold-start case), the Strategy context selects `TrendingRecommendationStrategy`, surfacing currently popular apps. If the user has download history, it switches to `TopRatedRecommendationStrategy`, which favors highly rated apps and a category-growth weighting so that, for example, a fast-growing category like Gaming or Productivity is slightly favored over a flatter one like Utilities. The AppBot chatbot uses a separate, conversational prompt path: free-text user queries are sent to Gemini along with a constrained instruction to respond with a short list of relevant app names, which the backend then resolves back into real marketplace listings before returning them to the frontend.

8.5 Moderation & Analytics Endpoints

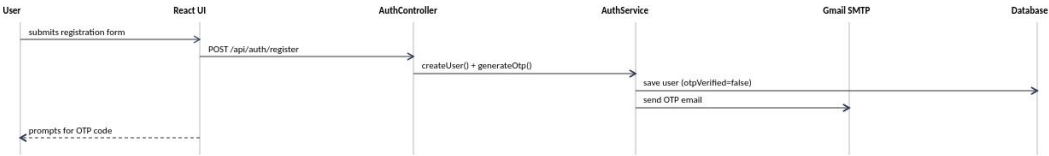
- Apps move through PENDING → APPROVED / REJECTED; only APPROVED apps with `isVisible = true` are returned by the public marketplace listing endpoints.
- `getSentimentAnalysis(appId)` returns a sentiment distribution map (counts and percentages) for a single app's reviews, used to power the per-app “AI Sentiment” tab.
- `getCategoryInsights()` and `getTrendingAnalysis()` back the platform-wide analytics shown to admins, including the fastest-growing category and current active recommendation strategy.
- `predictDownloads(appId)` projects 30-day download growth for a given app using its current download count, average rating, and category growth coefficient — a lightweight rule-based model rather than a trained ML model, intentionally kept transparent and explainable.

9. Key Workflow Sequences

The three sequence diagrams below trace the most representative end-to-end flows in the system: account registration with OTP verification, review submission with AI sentiment scoring, and the developer-publish / admin-approve lifecycle.

AppVerse AI — Key Workflow Sequences

A. Registration & Gmail OTP Verification



B. Review Submission with AI Sentiment Analysis



C. Developer Publish & Admin Approval

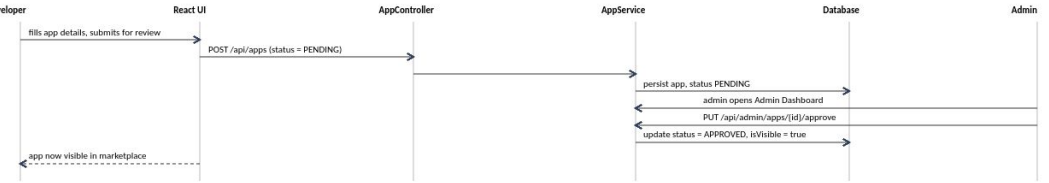


Fig. 7.1: Sequence-style workflow diagrams for the three most representative end-to-end flows in AppVerse AI.

Fig. 9.1: Sequence diagrams for registration, review submission, and the publish-approve workflow.

9.1 Registration & Gmail OTP Verification

A new user submits the registration form; the backend creates the account in an unverified state and generates a one-time code, which is emailed via Gmail SMTP. The frontend then prompts for the OTP code, and the account is only activated once the correct code is confirmed against the stored value.

9.2 Review Submission with AI Sentiment Analysis

After rating and writing a review, the request reaches ReviewService, which calls the Gemini API (or the rule-based fallback analyzer) to classify sentiment and predict a rating from the text. The fake-review

heuristic runs immediately afterward, and the review, along with all AI-derived fields, is persisted in a single transaction that also recalculates the app's average rating.

9.3 Developer Publish & Admin Approval

A developer's submission is stored with PENDING status and is not visible to end users. It appears in the Admin Dashboard's pending-apps queue, where an admin can preview the listing before approving or rejecting it. Approval flips the app's status and visibility flags in the same transaction, making it immediately discoverable in the public marketplace.

10. Module Descriptions

10.1 Authentication Module

Handles registration, Gmail OTP verification, login, and JWT issuance. Ensures every account is tied to a real, verified email address before it can be used, and that every subsequent request is authenticated statelessly via Bearer token.

10.2 Marketplace Module

Provides search, category filtering, and listing detail pages for end users. Tracks per-user download state so the UI can distinguish apps already downloaded from new ones, and feeds download events into the analytics and recommendation pipelines.

10.3 Review & Sentiment Module

Owns the full lifecycle of a review: submission, AI sentiment classification, rating prediction, fake-review flagging, editing, and deletion (with ownership checks so only the original author can edit or remove their own review). Also recalculates the parent app's aggregate rating whenever the visible review set changes.

10.4 Developer Console Module

Gives developers a dedicated space to create and edit apps, publish new versions with release notes, and track real download counts and ratings for everything they own. The Analytics tab within the console visualizes downloads over time, revenue by app, and per-app engagement.

10.5 Admin Module

Centralizes platform governance: a single dashboard surfaces pending app submissions and flagged reviews side by side, so an admin can action both quality concerns from one screen. Approve/Reject decisions on apps and moderation decisions on reviews are both logged through the same ModerationStatus-driven workflow.

10.6 AI Recommendation Module

Implements the Strategy-pattern recommendation engine described in Section 2.3 and Section 8.4, along with the trending-analysis and category-insight aggregations used on both the user marketplace and the admin analytics views.

10.7 AppBot Chatbot Module

A conversational layer on top of the Gemini API that lets users ask open-ended questions about what apps might suit their needs, returning structured, clickable app suggestions rather than free-form text.

11. Project Screenshots

The screenshots below were captured directly from the running application and illustrate the user, developer, and admin experiences described throughout this document.

11.1 AppBot AI Chatbot

The AppBot widget is available across the marketplace. Users can ask free-text questions (“Suggest a few apps which will be helpful for learning”) and receive Gemini-generated, clickable app recommendations directly inside the chat panel.

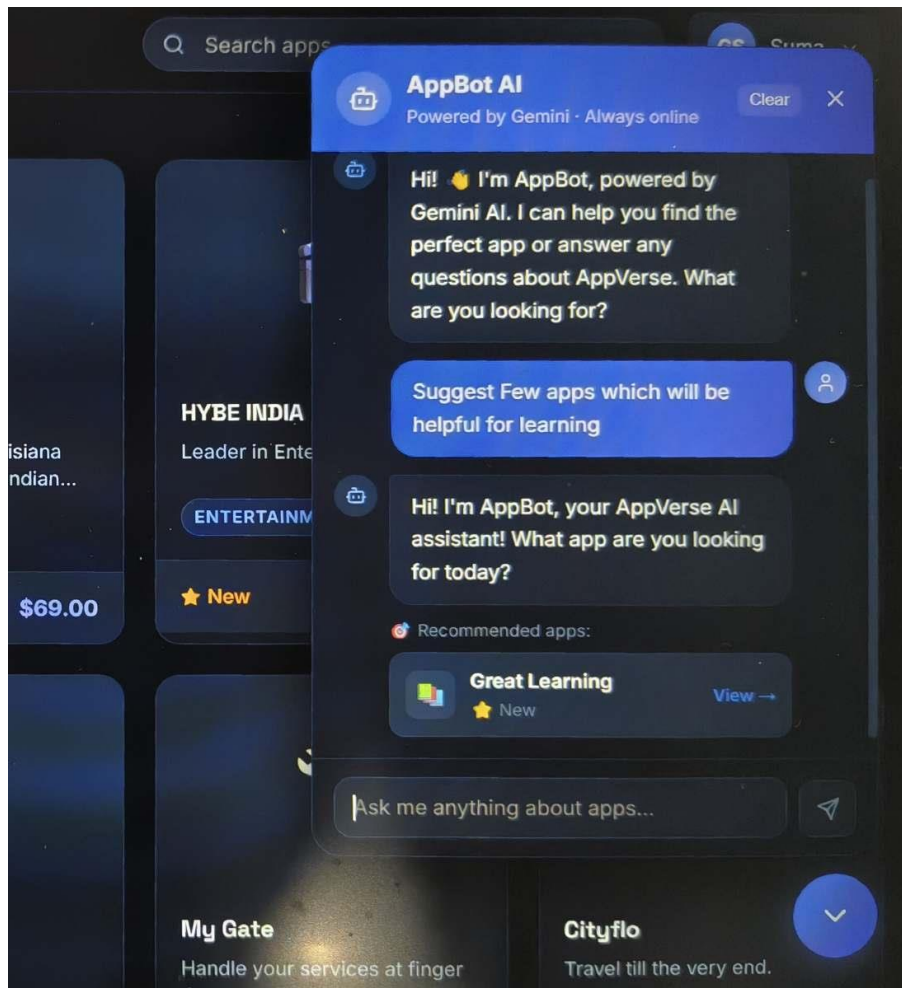


Fig. 11.1: AppBot AI assistant responding to a natural-language request with a recommended app card.

11.2 Marketplace & Category Browsing

The public marketplace lets users filter apps by category. Here the Shopping and Education categories are shown, including the Flipkart listing used as a running example throughout this document.

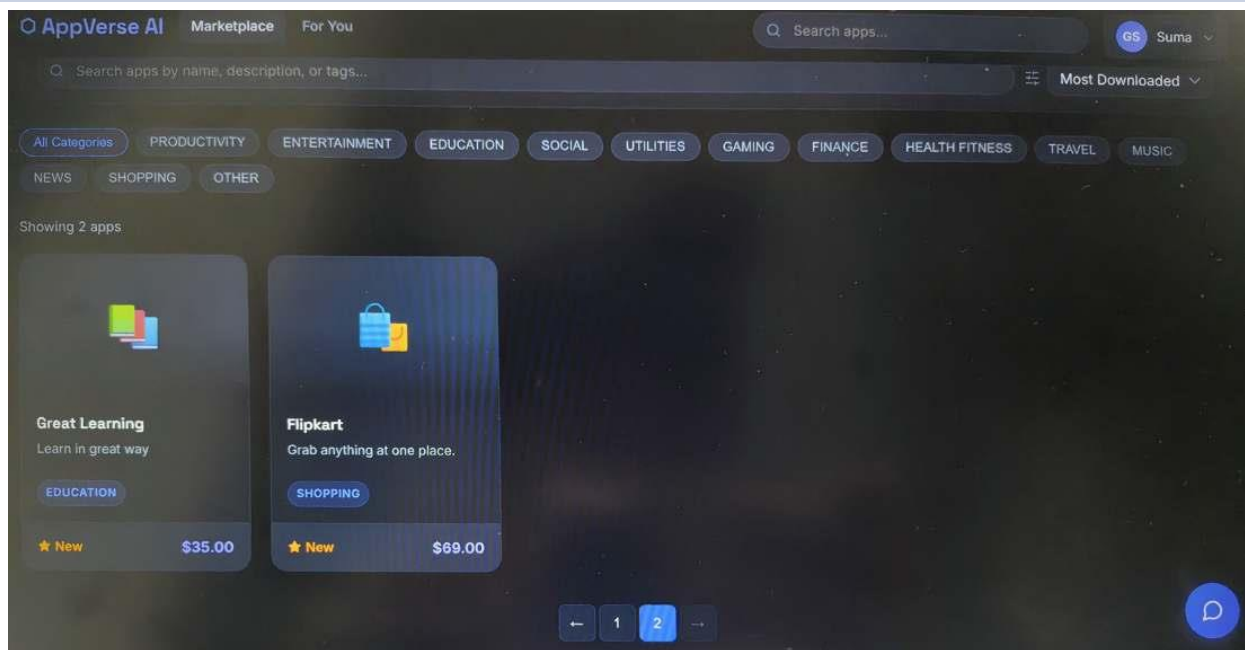


Fig. 11.2: Marketplace view filtered to Shopping/Education, showing the Flipkart and Great Learning listings.

11.3 Publishing a New App

Developers submit new listings through a structured form capturing the app name, category, tagline, description, price, version, and release notes before the listing is sent for admin review.

The screenshot shows a dark-themed submission form for an app named 'Flipkart'. The form includes the following fields and content:

- App Name ***: Flipkart
- Category ***: SHOPPING (with a dropdown arrow)
- Tagline**: Grab anything at one place.
- Description ***: In Flipkart, you can shop, explore and purchase every kind of sectore products coming from healthcare to automotive etc. (Note: 'sectore' is misspelled and underlined in red in the image).
- Price (USD)**: 69
- Version**: 4.7.2
- Tags**: ai, shopping
- Release Notes**: In this new version, an update is included which suggests customers the suitable products for them based on their previous purchases whichlist. You can search product category wise. (Note: 'whichlist' is misspelled and underlined in red in the image).
- Buttons**: Cancel and Submit for Review.

Fig. 11.3: New app submission form, shown here for the sample listing “Flipkart.”

11.4 Admin Approval Workflow

Pending submissions appear in the Admin Dashboard's queue, where an administrator can preview the listing and either approve it — making it instantly visible in the marketplace — or reject it.

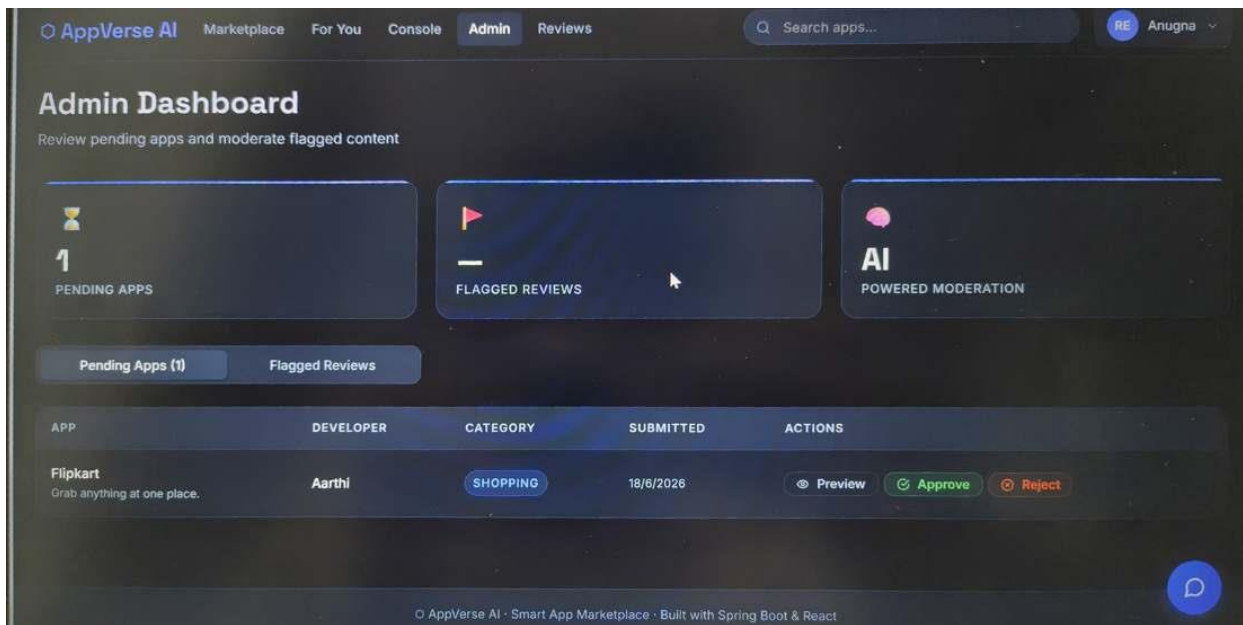


Fig. 11.4: Admin Dashboard — pending app awaiting Approve / Reject decision.

11.5 App Detail Page with AI Sentiment & Similar Apps

On an individual app's detail page, reviews carry an AI-derived sentiment tag (e.g. NEUTRAL), and a “Similar Apps” panel — generated by the recommendation service — surfaces related listings alongside the AI Sentiment tab.

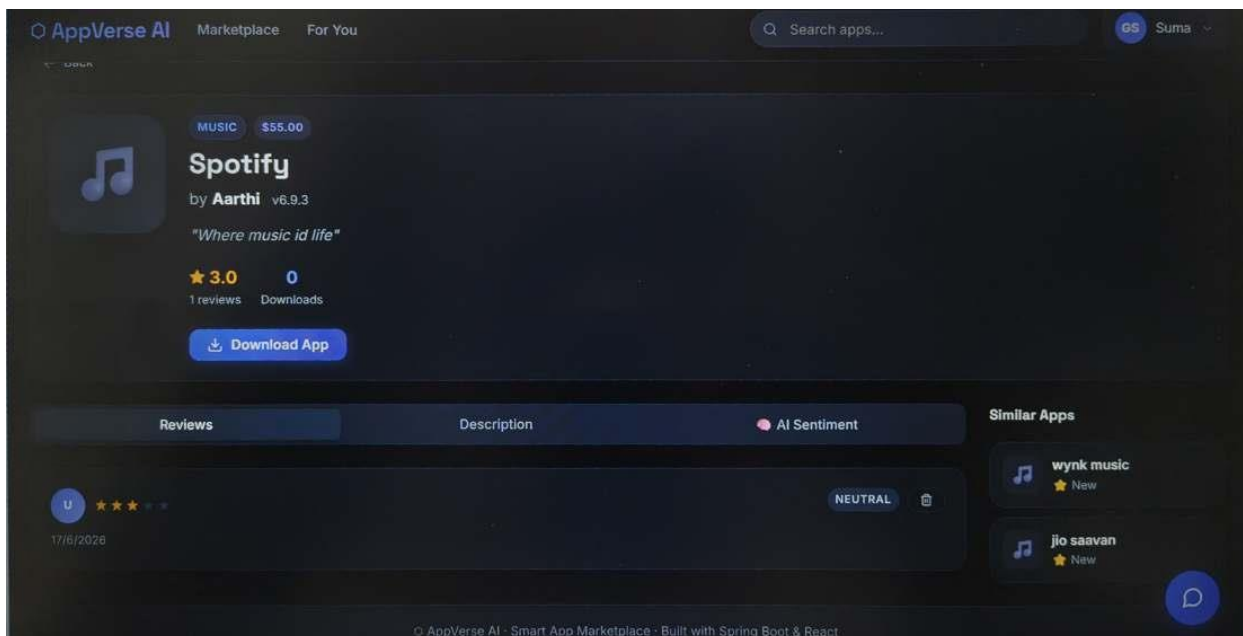


Fig. 11.5: App detail page (Spotify) with review sentiment tag and an AI-curated Similar Apps panel.

11.6 Developer Console — My Apps

The Developer Console's My Apps tab lists every app owned by the signed-in developer with its category, live status, download count, and rating, alongside quick actions to view, edit, track analytics, or remove a listing.

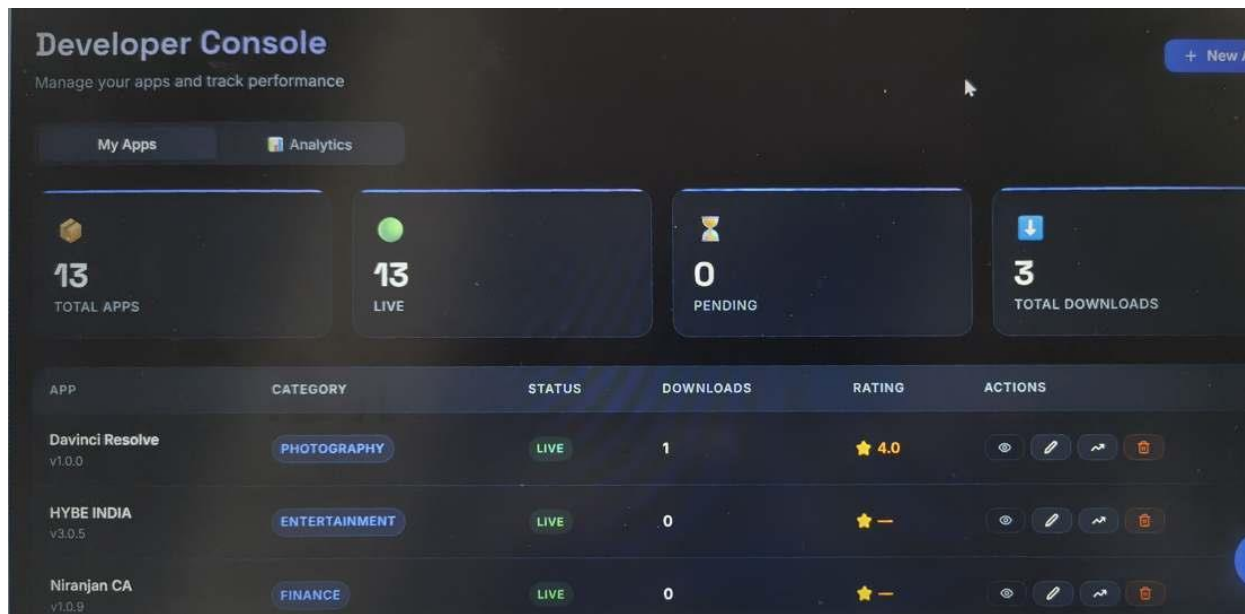


Fig. 11.6: Developer Console — My Apps overview with status, downloads, and rating per app.

11.7 Developer Analytics — Downloads Trend

Developers can track download trends for their own apps over time directly from the console, alongside total downloads, revenue, reviews, and average rating summary cards.

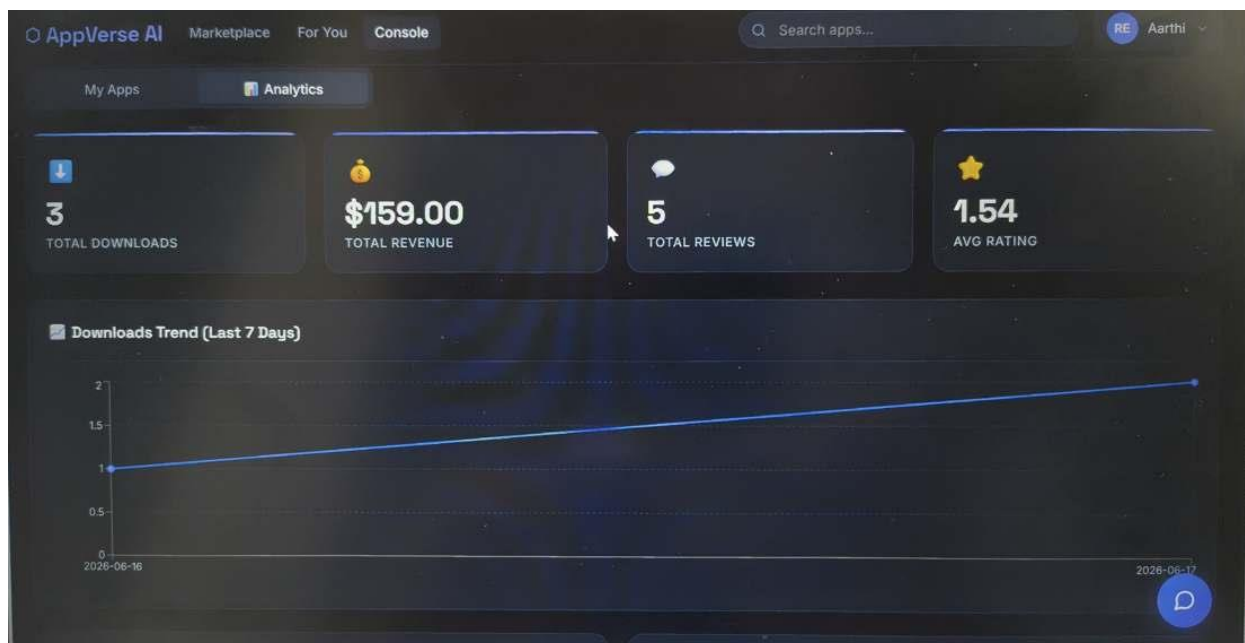


Fig. 11.7: Developer Console — Analytics tab showing total downloads, revenue, reviews, and average rating.

11.8 Platform & Revenue Analytics

Administrators and developers have access to richer analytics views, including revenue generated per app, the overall split between free and paid listings, and a per-app engagement report.

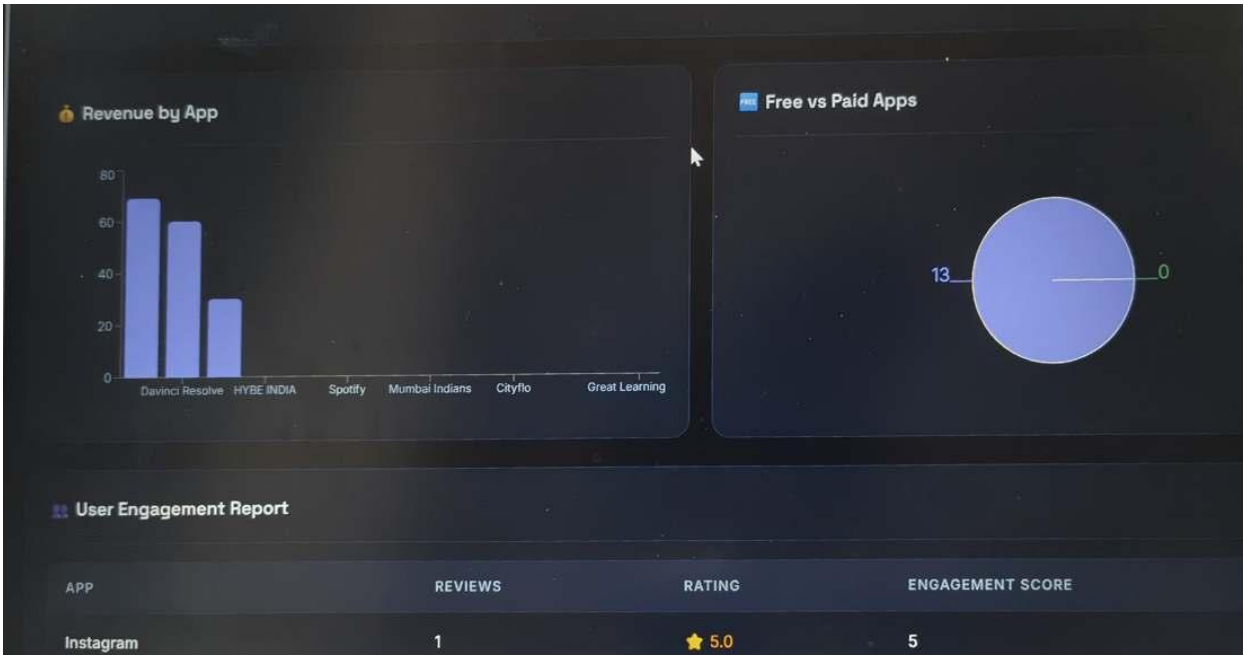


Fig. 11.8: Revenue-by-app chart, free-vs-paid distribution, and the user engagement report.

12. Deployment & Submission Guidelines

The submission package for AppVerse AI is organized into three packs, consistent with the program's standard capstone submission format.

12.1 Engineering Pack

1. Source Code — complete Spring Boot backend and React frontend codebases
2. Database Scripts — MySQL schema creation and seed scripts
3. Environment Files — application.properties / .env templates (with secrets redacted)
4. API Collections — Postman collection covering authentication, marketplace, review, recommendation, and admin endpoints

12.2 Documentation Pack

5. Project Report — this document
6. Architecture Diagrams — Section 2.1
7. ER Diagrams — Section 3.2
8. User Manual — role-based walkthrough (Sections 7, 10, 11)
9. Deployment Guide — Section 12.3

12.3 Validation Pack

10. Postman Collection — included in the Engineering Pack
11. Testing Evidence — JUnit test classes for authentication, app, and review services
12. Screenshots — Section 11
13. Execution Logs — captured during local test runs

12.4 Running the Project Locally

- Backend: import the Maven project into Spring Tool Suite 4, configure application.properties with a local MySQL connection string and a Gemini API key, then run as a Spring Boot application.
- Database: execute the provided schema script against a local MySQL instance before first run; Hibernate is configured to validate (not auto-update) the schema in this build.
- Frontend: install dependencies and start the React development server, pointing its API base URL at the locally running backend.

13. Conclusion & Future Enhancements

13.1 Conclusion

AppVerse AI successfully delivers a complete, working app marketplace that goes beyond standard CRUD functionality by integrating real generative-AI features into its core workflows. Across the three capstone-relevant pillars — authentication and role management, the marketplace and developer publishing pipeline, and AI-driven review intelligence and recommendations — the system demonstrates a clean layered backend, a role-aware frontend, and the deliberate use of object-oriented design patterns to keep the codebase maintainable. The moderation workflow ensures both app quality and review quality are gated by a human decision, while the AppBot assistant and recommendation engine show how an LLM can be embedded into a transactional application without compromising the predictability of the system's core business logic.

13.2 Lessons Learned

Building AppVerse AI surfaced several practical considerations worth recording. Designing prompts for the Gemini API that reliably return parseable, structured output required iterating on explicit formatting instructions and adding defensive parsing on the backend. Separating AI-derived fields (sentiment, predicted rating, fake-review flag) from user-submitted fields at the entity level made it much easier to reason about moderation logic and to avoid ever trusting client input for fields that should only ever be system-computed. Finally, applying the Strategy pattern to recommendations — rather than hard-coding an if/else inside the service — made it straightforward to add a second recommendation algorithm without touching any existing calling code.

13.3 Future Enhancements

- Replace the rule-based fake-review heuristic with a model trained on a labelled review dataset for higher precision.
- Add real-time notifications (email or in-app) when a developer's app is approved, rejected, or receives a new flagged review.
- Extend the recommendation engine with a third, collaborative-filtering-based strategy for users with substantial download history.
- Add pagination and caching to the marketplace listing endpoint as catalog size grows.
- Introduce automated integration tests around the Gemini API calls using a mocked AI client, to keep CI runs independent of external API availability.

14. Submitted By

Name: Gopidi Anugna Reddy

Project: AppVerse AI — Smart App Marketplace & Review Intelligence Platform

Under Guidance Of: Javeed Sir

Training Platform: Great Learning

Organization: Wipro

Date: June 2026

*AppVerse AI — Smart App Marketplace & Review Intelligence Platform
Wipro Capstone Project · Version 1.0.0 · June 2026*